AI Lab assignment 8

Tejas Patil
U24AI061

Q1)

```cpp
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
#include <random>
#include <climits>
#include <iomanip>
#include <map>
#include <numeric>

using namespace std;

// --- CONFIGURATION ---
// Use constexpr for compile-time constants to avoid "expression must have
a constant value"
constexpr int N = 8;

const string cities[N] = {"A", "B", "C", "D", "E", "F", "G", "H"};

const int cost_matrix[N][N] = {
    {0, 10, 15, 20, 25, 30, 35, 40},
    {12, 0, 35, 15, 20, 25, 30, 45},
    {25, 30, 0, 10, 40, 20, 15, 35},
    {18, 25, 12, 0, 15, 30, 20, 10},
    {22, 18, 28, 20, 0, 15, 25, 30},
    {35, 22, 18, 28, 12, 0, 40, 20},
    {30, 35, 22, 18, 28, 32, 0, 15},
    {40, 28, 35, 22, 18, 25, 12, 0}
};

mt19937 gen(random_device{}());

// --- HELPER FUNCTIONS ---
```

```
int calculate_cost(const vector<int>& tour) {
    int total_cost = 0;
    for (int i = 0; i < N - 1; ++i) {
        total_cost += cost_matrix[tour[i]][tour[i+1]];
    }
    total_cost += cost_matrix[tour.back()][tour[0]]; // return to start
    return total_cost;
}

vector<int> random_tour() {
    vector<int> tour(N);
    iota(tour.begin(), tour.end(), 0);
    shuffle(tour.begin(), tour.end(), gen);
    return tour;
}

vector<vector<int>> generate_neighbors(const vector<int>& tour) {
    vector<vector<int>> neighbors;
    for (int i = 0; i < N; ++i) {
        for (int j = i + 1; j < N; ++j) {
            vector<int> next_tour = tour;
            swap(next_tour[i], next_tour[j]);
            neighbors.push_back(next_tour);
        }
    }
    return neighbors;
}

// --- LOCAL BEAM SEARCH ---

int local_beam_search(int k, int max_iterations = 100) {
    cout << "\nRunning Local Beam Search with k = " << k << endl;

    vector<vector<int>> beam;
    for (int i = 0; i < k; ++i) beam.push_back(random_tour());

    int best_overall_cost = INT_MAX;
    vector<int> best_overall_tour;
    int iteration = 0;
```

```
    while (iteration < max_iterations) {
        iteration++;
        vector<vector<int>> all_neighbors;

        for (const auto& state : beam) {
            auto neighbors = generate_neighbors(state);
            all_neighbors.insert(all_neighbors.end(), neighbors.begin(),
neighbors.end());
        }

        // Sort neighbors by cost (Ascending)
        sort(all_neighbors.begin(), all_neighbors.end(), [](const
vector<int>& a, const vector<int>& b) {
            return calculate_cost(a) < calculate_cost(b);
        });

        // Select the top k unique neighbors for the next beam
        beam.clear();
        for (int i = 0; i < (int)all_neighbors.size() && (int)beam.size()
< k; ++i) {
            beam.push_back(all_neighbors[i]);
        }

        int current_best_cost = calculate_cost(beam[0]);
        cout << "Iteration " << iteration << " | Best Cost: " <<
current_best_cost << endl;

        if (current_best_cost < best_overall_cost) {
            best_overall_cost = current_best_cost;
            best_overall_tour = beam[0];
        } else {
            cout << "Converged." << endl;
            break;
        }
    }

    cout << "Final Cost: " << best_overall_cost << endl;
    return best_overall_cost;
}
```

```
int main() {
    vector<int> k_values = {3, 5, 10};
    map<int, int> results;

    for (int k : k_values) {
        results[k] = local_beam_search(k);
    }

    // Final Table Output

    cout << "\n" << string(60, '=') << endl;
    cout << left << setw(10) << "k" << setw(20) << "Speed" << "Quality
(Cost)" << endl;
    cout << string(60, '-') << endl;

    cout << left << setw(10) << "3" << setw(20) << "Fast" << results[3] <<
endl;
    cout << left << setw(10) << "5" << setw(20) << "Moderate" <<
results[5] << endl;
    cout << left << setw(10) << "10" << setw(20) << "Slower" <<
results[10] << endl;

    return 0;
}
```

```
Running Local Beam Search with k = 3
Iteration 1 | Best Cost: 149
Iteration 2 | Best Cost: 125
Iteration 3 | Best Cost: 123
Iteration 4 | Best Cost: 124
Converged.
Final Cost: 123

Running Local Beam Search with k = 5
Iteration 1 | Best Cost: 132
Iteration 2 | Best Cost: 132
Converged.
Final Cost: 132

Running Local Beam Search with k = 10
Iteration 1 | Best Cost: 132
Iteration 2 | Best Cost: 123
Iteration 3 | Best Cost: 121
Iteration 4 | Best Cost: 121
Converged.
Final Cost: 121
```

```
============================================================
k         Speed               Quality (Cost)
------------------------------------------------------------
3         Fast                123
5         Moderate            132
10        Slower              121
```

Q2)

```cpp
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
#include <random>
#include <ctime>
#include <iomanip>

using namespace std;

// --- GLOBAL SETTINGS ---
const int N = 8;
const string CITIES[N] = {"A", "B", "C", "D", "E", "F", "G", "H"};

const int COST_MATRIX[N][N] = {
    {0, 10, 15, 20, 25, 30, 35, 40},
    {12, 0, 35, 15, 20, 25, 30, 45},
    {25, 30, 0, 10, 40, 20, 15, 35},
    {18, 25, 12, 0, 15, 30, 20, 10},
    {22, 18, 28, 20, 0, 15, 25, 30},
    {35, 22, 18, 28, 12, 0, 40, 20},
    {30, 35, 22, 18, 28, 32, 0, 15},
    {40, 28, 35, 22, 18, 25, 12, 0}
};

// --- RANDOM GENERATOR ---
mt19937 rng(time(0));

// --- HELPER FUNCTIONS ---

int calculate_cost(const vector<int>& tour) {
    int total = 0;
    for (int i = 0; i < N - 1; i++) {
        total += COST_MATRIX[tour[i]][tour[i+1]];
    }
    total += COST_MATRIX[tour[N-1]][tour[0]];
    return total;
}
```

```cpp
bool already_in_tour(const vector<int>& tour, int city) {
    for (int x : tour) {
        if (x == city) return true;
    }
    return false;
}

// Better shuffle using mt19937
vector<int> get_random_tour() {
    vector<int> tour;
    for (int i = 0; i < N; i++) tour.push_back(i);
    shuffle(tour.begin(), tour.end(), rng);
    return tour;
}

// --- SELECTION (Tournament) ---
vector<int> tournament_selection(const vector<vector<int>>& population) {
    int k = 3; // tournament size
    vector<int> best;
    int best_cost = INT_MAX;

    for (int i = 0; i < k; i++) {
        int idx = rng() % population.size();
        int cost = calculate_cost(population[idx]);
        if (cost < best_cost) {
            best_cost = cost;
            best = population[idx];
        }
    }
    return best;
}

// --- CROSSOVER ---
vector<int> crossover(const vector<int>& p1, const vector<int>& p2) {
    int start = rng() % N;
    int end = rng() % N;
    if (start > end) swap(start, end);

    vector<int> child(N, -1);
```

```
    // Copy segment from parent1
    for (int i = start; i <= end; i++) {
        child[i] = p1[i];
    }

    // Fill remaining from parent2
    int idx = 0;
    for (int i = 0; i < N; i++) {
        if (!already_in_tour(child, p2[i])) {
            while (child[idx] != -1) idx++;
            child[idx] = p2[i];
        }
    }

    return child;
}

// --- MUTATION ---
void mutate(vector<int>& tour) {
    if ((rng() % 100) < 20) { // 20% mutation
        int i = rng() % N;
        int j = rng() % N;
        swap(tour[i], tour[j]);
    }
}

// --- PRINT TOUR ---
void print_tour(const vector<int>& tour) {
    for (int i : tour) {
        cout << CITIES[i] << " ";
    }
    cout << CITIES[tour[0]]; // return to start
    cout << endl;
}

// --- MAIN GA ---
void run_genetic_algorithm(int generations, int pop_size) {
    vector<vector<int>> population;

    // Initial population
```

```
    for (int i = 0; i < pop_size; i++) {
        population.push_back(get_random_tour());
    }

    vector<int> best_tour;
    int best_cost = INT_MAX;

    cout << left << setw(10) << "Gen" << "Best Cost" << endl;
    cout << "----------------------" << endl;

    for (int g = 0; g <= generations; g++) {

        vector<vector<int>> next_gen;

        // --- FIND BEST (ELITISM) ---
        for (auto& tour : population) {
            int cost = calculate_cost(tour);
            if (cost < best_cost) {
                best_cost = cost;
                best_tour = tour;
            }
        }

        // Keep best solution
        next_gen.push_back(best_tour);

        // --- CREATE NEXT GENERATION ---
        while (next_gen.size() < pop_size) {
            vector<int> parent1 = tournament_selection(population);
            vector<int> parent2 = tournament_selection(population);

            vector<int> child = crossover(parent1, parent2);
            mutate(child);

            next_gen.push_back(child);
        }

        population = next_gen;

        if (g % 10 == 0) {
```

```
            cout << left << setw(10) << g << best_cost << endl;
        }
    }

    // --- FINAL RESULT ---
    cout << "\nBest Tour Found:\n";
    print_tour(best_tour);
    cout << "Minimum Cost: " << best_cost << endl;
}

// --- MAIN ---
int main() {
    run_genetic_algorithm(100, 50);
    return 0;
}
```

```
Gen        Best Cost
----------------------
0          132
10         121
20         121
30         121
40         121
50         121
60         121
70         121
80         121
90         121
100        121

Best Tour Found:
C D H G F E B A C
Minimum Cost: 121
```